Saarthi Live Final Design Workflow
Peacock feather logo: assets/peacock-feather-original.png

Product Goal
Saarthi Live gives users a multilingual AI conversation experience that can shift between a general assistant, resume-led interview, Hindi resume creation, and candidate profile analysis without mixing unrelated memories across flows.

User-Facing Modes
Home
  -> General AI Assistant
  -> Upload Resume -> Resume-led Live Interview
-> Hindi Consultant Resume Builder -> Download Hindi and English Resume -> Resume-led Live Interview
  -> Candidate Profile
General AI Assistant -> Candidate Profile
Resume-led Live Interview -> Candidate Profile

End-To-End System Flow
User speaks or types
  -> Expo App: Web or Mobile
  -> FastAPI Route
  -> Application Service
  -> Security and Audit Service
  -> Agent Orchestrator
  -> Conversation Graph
  -> STT Service Interface -> Sarvam STT
  -> LLM Service Interface -> Sarvam-m
  -> TTS Service Interface -> Sarvam TTS
  -> Typed API Response
  -> Expo App

Backend SOLID Shape
api/routes_*.py
  -> services/application.py
      -> VoiceApplicationService
      -> ResumeApplicationService
      -> ProfileApplicationService
      -> SystemApplicationService
      -> LiveKitTokenApplicationService

VoiceApplicationService
  -> AgentOrchestrator
      -> conversation_graph.py
      -> LLMService Protocol
      -> SpeechToTextService Protocol

-> AuditService Protocol

ResumeApplicationService
-> LLMService Protocol
-> DocumentParserService Protocol
-> UploadSecurityService Protocol
-> AuditService Protocol

Provider Implementations
LLMService / STT / TTS -> services/sarvam.py
DocumentParserService -> services/document_parser.py
Audit and Upload Security -> services/governance.py
Structured Agents -> pydantic_agents.py
Conversation State Machine -> conversation_graph.py

Interview State Machine
resume_loaded -> interview_started -> followup_question -> profile_signal_update ->
close_session

Resume Upload Flow
User uploads or pastes resume -> POST /resume/analyze -> ResumeApplicationService.analyze ->
DocumentParserService extracts text -> ResumeAnalyzerAgent creates structured prompt ->
LLMService calls Sarvam-m -> ResumeAnalyzerAgent parses output -> ResumeAnalyzeResponse
returns to app -> Start Interview

Hindi Resume Builder Flow
User speaks or enters local profile details -> POST /resume/build ->
ResumeApplicationService.build -> ResumeBuilderAgent creates bilingual resume prompt ->
LLMService calls Sarvam-m -> ResumeBuilderAgent parses Hindi and English resume ->
ResumeAnalyzeResponse returns to app -> Download resume or start interview

Candidate Profile Flow
Conversation/interview turns -> POST /candidate/profile -> ProfileApplicationService ->
CandidateProfileAgent -> LLMService calls Sarvam-m -> CandidateProfileResponse returns to
Candidate Profile page

Memory Boundaries
General Assistant starts with fresh general memory.
Resume-led interview starts with fresh interview memory.
Resume context is carried into interview mode.
Previous general assistant turns do not leak into interview history.
Candidate Profile uses the current active discussion/interview memory.

Security And Governance
Incoming request -> require_actor -> optional API token -> optional consent gate -> rate
limiting -> upload allowlist and size checks -> unsafe marker screen -> PII redacted logs ->

metadata audit JSONL -> prompt-injection cleanup

Current Limitations

Expo Go uses short audio chunks, not native LiveKit streaming.
Full LiveKit realtime voice requires a development build.
No encrypted database because conversations are not persisted.
No external malware scanner yet.
No full identity provider/RBAC console yet.